
Aajoo ⇌ BotPenguin — API reference

Every endpoint the chatbot can call, what it must send, and what comes back. Read off the live routing table on 6 October 2026.

Aajoo Homes — BotPenguin integration

Prepared 6 October 2026 by the ZypheX Tech development team

Verified against the live production backend (api.aajoohomes.com)

For: the BotPenguin team, and anyone wiring a chat flow to Aajoo **Date:** 6 October 2026 · **Endpoints:** 28 · **Base URL:** `https://api.aajoohomes.com`

Everything the chatbot can ask our backend for, what it must send, and what comes back. Taken from the live routing table and controller, not from an older design document.

1. How BotPenguin authenticates

Every bot-facing endpoint requires one header:

```
x-botpenguin-token: <shared token>
```

Where the value lives	Render environment variable <code>BOTPENGUIN_API_TOKEN</code>
How you get it	Sent to you separately — never in this document, a repository, or a chat message
Wrong or missing	<code>401 Unauthorized</code>
Not configured on our side	<code>500 BOTPENGUIN_API_TOKEN is not configured in env</code>

Three endpoints use a different key, and each is called out where it appears: the two `/bp/export/*` endpoints use an **admin** token, and `/bp/handoff` is called by the **visitor's browser** with their own login rather than by BotPenguin.

2. The envelope

Every response has the same two-layer shape. The payload is always under `data`:

```
{ "success": true, "data": { "...": "..." } }
```

Errors keep the shape and carry a message:

```
{ "success": false, "message": "Session not found" }
```

Common statuses: `400` bad input · `401` bad or missing token · `403` session not identified, or needs OTP · `404` no session, or no such record **for this account** · `429` rate limited · `500` our side.

3. The session is the thread of identity

Almost every call takes `session_id`. Open one with `/bp/session/start`; everything after it is answered in the context of that session.

This matters for a reason worth stating plainly: **we do not trust record ids sent by the bot**. Where a request names a `booking_id`, `host_id` or `property_id`, we check that it belongs to the account the session is linked to. A session not linked to a verified account is refused rather than served.

Three consequences for your flows:

1. **An anonymous session can browse, not retrieve.** Search and public property data work; bookings, payouts and invoices do not.
2. **Money and private detail need a recent OTP.** A verification lasts **30 minutes**, after which the bot must ask again.
3. **Failures are deliberately vague.** "No such record for this account" comes back whether the record is missing or simply someone else's, so nobody can discover valid booking ids by watching which error returns.

4. Endpoints

Session

Method	Path	What it does
POST	<code>/bp/session/start</code>	Opens a chat session and returns its <code>session_id</code>
POST	<code>/bp/session/get-by-phone</code>	Finds an existing session for a phone number
POST	<code>/bp/session/analyze</code>	Scores a message for intent, sentiment and urgency
POST	<code>/bp/context</code>	Who this session is, and what it may do

`/bp/session/start` Send — `phone`, `channel_type` (`web` / `whatsapp` / ...), `user_role`, `language`, and `token` (an optional handoff token, see below) Returns — `session_id`, `authenticated`, `token_status` (`absent` / `valid` / `expired` / `invalid`), `user_name`, `user_role`, `phone`, `language`

`/bp/session/get-by-phone` Send — `phone`, `channel_type` · Returns — `session_id`, `user_role`, `language`, `channel_type`, `message`

`/bp/session/analyze` Send — `session_id`, `message`, `category` · Returns — `reply_text`, `intent`, `sentiment_score`, `sentiment_label`, `urgency_score`, `urgency_label`, `needs_escalation`

`/bp/context` Send — `session_id` · Returns — `user_role`, `user_id`, `otp_verified`, and the session's stored context. Use this to decide what to offer *before* offering it.

Listings and property

Method	Path	What it does
POST	<code>/bp/listings/search</code>	Search stays by city and dates
POST	<code>/bp/property/location</code>	Where a property is

Method	Path	What it does
POST	<code>/bp/property/amenities</code>	What a property offers

`/bp/listings/search` Send — `session_id`, `city`, `checkin`, `checkout`, `guests`, `page`, `limit` Returns — `listings_text` (ready to speak), `listings_status`, `prop_image`, `pagination`, `has_more`, `next_page`

`/bp/property/location` Send — `session_id`, `property_id` · Returns — `location_text`, `detail_level`, `note`

`/bp/property/amenities` Send — `session_id`, `property_id` · Returns — `amenities_text`, `detail_level`, `wifi_name`, `wifi_password`

`detail_level` is the **privacy switch**. The exact address and the wi-fi credentials are released only to a guest with a confirmed stay at that property. Everyone else gets the general area and no credentials. Do not cache these fields across sessions.

Booking

Method	Path	What it does
POST	<code>/bp/booking/details</code>	A booking's particulars
POST	<code>/bp/booking/policy</code>	Cancellation policy, and what a refund would be
POST	<code>/bp/booking/modify</code>	Request a change or a cancellation

All three take `session_id` and `booking_id` — or `booking_id_manual`, the `B...` code a guest reads out — and all three check the booking belongs to this session's account.

Endpoint	Returns
<code>/bp/booking/details</code>	<code>details_text</code> , <code>booking_id</code> , <code>property_id</code>
<code>/bp/booking/policy</code>	<code>policy_text</code> , <code>policy_name</code> , <code>policy_key</code> , <code>policy_terms</code> , <code>refund_eligibility</code> , <code>refund_eligibility_percent</code> , <code>refund_amount</code> , <code>manual_review</code>
<code>/bp/booking/modify</code>	<code>success</code> , <code>message</code> , <code>booking_context</code> , <code>booking_id</code> , <code>property_name</code> , <code>property_id</code> , <code>open_case_id</code> , <code>open_case_status</code> , <code>refund_text</code> , <code>refund_amount</code> , <code>refund_expected_by</code>

`/bp/booking/modify` also takes an `action`.

Identity (OTP)

Method	Path	What it does
POST	<code>/bp/otp/send</code>	Send a one-time code
POST	<code>/bp/otp/verify</code>	Check the code and privilege the session

`/bp/otp/send` — Send `session_id`, `otp_action` · Returns `success`, `verified`, `message` `/bp/otp/verify` — Send `session_id`, `otp` · Returns `verified`, `failure_attempts`, `message`

A verified code privileges the session for **30 minutes**. After that, the money and private-detail endpoints ask again.

Payment and documents

Method	Path	What it does
POST	<code>/bp/payment/check-transaction</code>	Did this payment go through?
POST	<code>/bp/payment/refund-status</code>	Where is my refund?
POST	<code>/bp/document/invoice</code>	The invoice for a stay

Endpoint	Send	Returns
<code>/bp/payment/check-transaction</code>	<code>session_id</code> , <code>booking_id</code> / <code>booking_id_manual</code>	<code>status_text</code> , <code>case_id</code> , <code>message</code>
<code>/bp/payment/refund-status</code>	<code>session_id</code> , <code>booking_id</code>	<code>refund_outcome</code> , <code>refund_amount</code> , <code>refund_percent</code> , <code>refund_text</code> , <code>expected_by</code>
<code>/bp/document/invoice</code>	<code>session_id</code> , <code>booking_id</code>	<code>invoice_text</code> , <code>invoice_status</code> , <code>invoice_url</code> , <code>cloudinary_invoice_url</code> , <code>invoice_note</code>

The invoice link is a **signed URL**. Treat it as short-lived and private — do not store it, and do not repeat it into another conversation.

Negotiation

Method	Path	What it does
POST	<code>/bp/negotiate</code>	Make the host a price offer

Send — `session_id`, `property_id`, `offer_price`, `book_from`, `book_to` Returns — the engine's own result, plus `message`.

This is **the same negotiation engine the website and the app use**, not a second copy of the rules. The guest is resolved from the session's phone and never taken from the request body: a bot that could name its own guest id could negotiate as anybody.

Support

Method	Path	Send	Returns
POST	<code>/bp/support/create-case</code>	<code>session_id</code> , <code>category</code> , <code>priority</code>	<code>case_id</code> , <code>category</code> , <code>sla</code> , <code>message</code>

Method	Path	Send	Returns
POST	/bp/support/close-case	session_id, case_id, confirmed	case_id, status, message
POST	/bp/support/housekeeping	session_id, request_type	case_id, sla (30 minutes), message
POST	/bp/support/log-event	session_id, type, data	message

category is checked against a fixed list — an unknown category is rejected rather than filed somewhere nobody is watching.

Host

Method	Path	Returns
POST	/bp/host/listing	listing_text, completion_percent
POST	/bp/host/calendar	calendar_text
POST	/bp/host/payout	payout_text, case_id
POST	/bp/host/analytics	analytics_text
POST	/bp/host/guest-issues	report_text, csv_url, total_issues

All take session_id and host_id, and **all verify the session belongs to that host**. A host id on its own proves nothing. /bp/host/guest-issues also takes user_id.

Not called by BotPenguin

POST /bp/handoff is called by the **visitor's own browser**, authenticated by their Aajoo login rather than the vendor token. It returns a **15-minute** handoff token that the web widget passes to /bp/session/start, so a signed-in visitor does not have to identify themselves again in chat. BotPenguin only ever forwards the token it is handed.

GET /bp/export/leads and GET /bp/export/logs are operations exports, guarded by x-admin-token — **not** the BotPenguin token. Query — from, to, plus status (leads) or type (logs). Return — count and data.

5. What the bot cannot do

Worth being explicit, because it shapes what a flow can promise a user:

- **It cannot take a payment.** No endpoint charges a card. The bot can report on a payment and hand over a link; the transaction happens on the website or in the app.
- **It cannot create or approve a booking.** It can request a modification, which joins the normal queue.
- **It cannot read across accounts.** Every record id is checked against the session's identity.

- **It cannot hold a privilege.** OTP verification expires after 30 minutes.

6. Rate limiting

Every endpoint sits behind a general rate limiter. On trip you get `429` — back off rather than looping, and show the user "give me a moment" rather than an error.

7. If something looks wrong

Send us the **endpoint path, the `session_id`, the time, and the full response body**. The `session_id` is what lets us find the exact exchange in our logs; without it, tracing a report takes far longer.

Please do not send us the shared token, an admin token, or a handoff token. They should exist only in the Render environment and in BotPenguin's own configuration.